

Resumen Ejecutivo

Sistema Computacional Integrado para Decisiones de Cadena de Suministro Industrial

Bernardo Sepúlveda Garza · Caso práctico — Supply Chain Analyst · Fecha de presentación: 9 de junio de 2026

El sistema resuelve de forma **integrada** las tres decisiones semanales que enfrenta una red de 5 almacenes que abastece a 20 puntos de demanda: **cuánto surtir** (optimización de distribución), **qué esperar y a quién proteger** (predicción de demanda y clasificación de riesgo de desabasto) y **cuándo alertar proactivamente** (agente autónomo de monitoreo). La arquitectura separa responsabilidades en módulos independientes (*optimization, ml_models, agent*) que comparten un mismo generador de datos sintéticos con semilla fija (**seed = 42**), de modo que toda la cadena es **100% reproducible** y ejecutable con un único script de entrada. La decisión de diseño transversal fue priorizar un sistema **determinista, trazable y verificable** por encima de la sofisticación: cada módulo expone funciones puras que el agente consume como herramientas, evitando dependencias de red o llaves de API.

1 · Optimización de la distribución

Decisión de diseño. Se modeló un **problema de transporte con capacidad limitada** y se resolvió con **programación lineal** usando **PuLP** sobre el solver **CBC**. Se eligió PuLP por su sintaxis declarativa, su solver de código abierto incluido (sin licencias) y su transparencia para auditar la formulación. La variable de decisión es el flujo continuo x_{ij} de unidades del almacén i al punto j ; el objetivo minimiza el costo total $\sum c_{ij} \cdot x_{ij}$ sujeto a dos familias de restricciones: **satisfacer la demanda** de cada punto — la suma de lo que llega a un punto j desde todos los almacenes debe cubrir su demanda d_j ($\sum_i x_{ij} \geq d_j$) — y **respetar la capacidad** de cada almacén — lo que sale de un almacén i no puede exceder su capacidad cap_i ($\sum_j x_{ij} \leq cap_i$). Se usó flujo continuo en lugar de entero porque las unidades logísticas son divisibles y la relajación lineal garantiza el óptimo global en tiempo despreciable.

Análisis de sensibilidad. El modelo se re-resuelve con la demanda incrementada **+20%** para cuantificar la robustez de la red ante un choque de demanda. La holgura de capacidad del 29.9% absorbe el choque sin infactibilidad, y el costo escala de forma **casi proporcional** (+23.0%), lo que indica una red bien dimensionada sin cuellos de botella estructurales.

Indicador	Escenario base	Estrés (+20%)
Costo total óptimo de distribución	\$43,181.00	\$53,130.00
Demanda total / Capacidad total	3,434 / 4,461 u	4,121 / 4,461 u
Holgura de capacidad sobre demanda	+29.9%	+8.3%
Variación de costo vs. base	—	+\$9,949 (+23.0%)

Costo de transporte por ruta en el rango \$5–\$49/unidad (media \$27.5, σ 13.5). El solver alcanza el óptimo global de forma determinista y la solución se visualiza como mapa de flujos y gráficas de costo.

2 · Predicción de demanda y clasificación de riesgo

2A · Pronóstico de demanda (regresión)

Sobre una serie sintética de 52 semanas con tendencia, estacionalidad y ruido, se construyeron *features* de **rezago (lags 1–3)** y se entrenaron **Random Forest** y **XGBoost**. La evaluación usa **validación walk-forward** (*TimeSeriesSplit*, 5 folds): el modelo sólo ve el pasado y se prueba sobre el futuro inmediato, **eliminando el data leakage** que produciría una validación cruzada aleatoria en datos temporales. Se compararon MAE, RMSE y MAPE. **Random Forest resultó el mejor modelo** y es el que consume el agente — la selección se hizo por **menor error de generalización bajo walk-forward**, no por ajuste en entrenamiento.

Modelo de pronóstico	MAE	RMSE	MAPE	Selección
Random Forest	16.62	18.98	10.21%	Elegido
XGBoost	> RF	> RF	> RF	Descartado

2B · Clasificación de riesgo de desabasto

Cómo se calculó el riesgo y por qué. Al no existir históricos reales de desabasto, la etiqueta se construyó con una **regla de negocio simulada** que reproduce las condiciones que elevan el riesgo logístico real: **bajo inventario, alta demanda, tiempos de entrega largos y grandes distancias**. El núcleo es un **índice de cobertura** (*coverage ratio*) que compara el inventario disponible contra el consumo esperado durante el *lead time*; cuando la cobertura es baja, el sistema interpreta mayor probabilidad de desabasto. El *target* binario marca como **riesgo alto** el 25% superior del score resultante.

Modelos y manejo del desbalance. Se entrenaron Random Forest y XGBoost sobre un dataset **desbalanceado 3:1** (150 bajo / 50 alto riesgo). En lugar de imponer una única técnica externa (por ejemplo, sobremuestreo del tipo SMOTE) a ambos modelos — lo que alteraría el dataset y ensuciaría la comparación —, se usó el **mecanismo nativo de cada algoritmo**: en Random Forest, la **ponderación de clases balanceada** reasigna el peso de cada clase en los cortes del árbol; en XGBoost, el **peso nativo de la clase positiva** corrige el gradiente durante el entrenamiento. Aprovechar el manejo interno de cada modelo mantiene los datos intactos, hace la comparación limpia y reproducible, y se evaluó con métricas robustas al desbalance (**F1 y ROC-AUC**, no *accuracy*). **XGBoost fue el mejor clasificador** (F1 = 0.812, AUC = 0.948) y es el que usa el agente para puntuar riesgo.

Clasificador de riesgo	F1-score	ROC-AUC	Manejo de desbalance
XGBoost	0.812	0.948	Peso nativo de clase — Elegido
Random Forest	< XGB	< XGB	Ponderación balanceada

2C · Interpretabilidad

Mediante *feature importance* de modelos de árbol (equivalente práctico a SHAP para este caso), la variable **más importante es el inventario (stock), seguida del lead time**, tanto en la predicción de riesgo como en su correlación con la demanda. Esto se confirma con la correlación negativa **stock vs. riesgo = -0.444** (la mayor en valor absoluto): **a menor inventario y mayor tiempo de reabasto, mayor probabilidad de desabasto**. Conclusión ejecutiva: la velocidad de reabastecimiento y el inventario disponible son las **palancas primarias** para gestionar el riesgo de suministro.

3 · Agente inteligente de monitoreo y alertas

El agente se construyó con una **implementación propia** en lugar de un framework como LangChain porque el razonamiento en cadena de suministro es **determinista** por naturaleza: las reglas de decisión son auditables y no requieren lenguaje natural. Esta decisión garantiza **reproducibilidad total y trazabilidad completa** del loop, y evita depender de **llaves de API** y servicios externos — lo que elimina **costos adicionales** recurrentes y la variabilidad de un LLM externo —, operando con **cero dependencias de red**.

La **integración de los módulos** funciona a través de un loop **ReAct** (Observar → Razonar → Actuar → Observar) sobre cuatro herramientas: *get_stock_status* consulta el inventario de los 5 almacenes; *get_demand_forecast* **consume el Random Forest del Módulo 2A** para proyectar demanda y el XGBoost del Módulo 2B para puntuar riesgo; *run_optimization* **invoca el solver del Módulo 1** ante un almacén crítico; y *send_alert* emite la notificación proactiva. Así, el agente es la **capa de integración** que une las tres decisiones en un solo flujo autónomo.

Sobre el **criterio de paro**, el agente observa los 20 puntos, clasifica su severidad (CRITICAL ≥ 0.75, HIGH 0.50–0.74) y actúa hasta que **todos los puntos en riesgo han sido atendidos**. Cada paso se exporta a *agent_trace.json* con herramienta, entradas, salidas, razonamiento y timestamp, dando **trazabilidad auditable** del proceso completo.

Resultado de la corrida del agente	Valor
Pasos de razonamiento registrados (trazabilidad)	34
Puntos en riesgo detectados y atendidos	6 (P1, P2, P4, P5, P8, P20)
Alertas proactivas emitidas	6 (5 CRITICAL, 1 HIGH)
Condición de paro alcanzada	Sí — todo riesgo atendido

4 · Ingeniería, reproducibilidad y estructura del proyecto

Separación de responsabilidades. El repositorio está organizado por responsabilidad (*data/*, *notebooks/*, *src/*, *tests/*), con un módulo por función: *data_generator* (datos sintéticos), *optimization* (Módulo 1), *ml_models* (Módulo 2), *agent* (Módulo 3), *visualization* y *utils*. Un único generador de datos alimenta a los tres módulos, de modo que optimización, pronóstico y agente operan siempre sobre la misma realidad sintética y sus resultados son comparables entre sí.

Reproducibilidad. Toda la cadena fija **seed = 42** — generación de datos, estado aleatorio de Random Forest y XGBoost, e inventarios simulados del agente —, por lo que cada ejecución produce resultados idénticos. Las **dependencias están fijadas a versión exacta** y el sistema corre de extremo a extremo desde un **único punto de entrada** que encadena los tres módulos en secuencia, de modo que cualquier integrante del equipo reproduce los mismos resultados con una sola ejecución.

Pruebas. Una suite de **12 pruebas unitarias** protege las invariantes críticas del sistema y respalda la confianza en los resultados reportados:

Archivo de pruebas	Tests	Qué valida
test_data_generator	4	Dimensiones, reproducibilidad y factibilidad del problema
test_optimization	3	Costo positivo, demanda satisfecha, estrés > base
test_agent	5	Detección de riesgo, alertas y herramientas del agente

Resultados clave

El sistema entrega, de extremo a extremo y de forma reproducible: (1) un **plan de distribución óptimo** de **\$43,181** que se mantiene factible ante un choque de demanda de +20% (costo +23.0%, sin infactibilidad); (2) un **pronóstico de demanda** con MAPE de **10.2%** (Random Forest, validado walk-forward) y un **clasificador de riesgo** con **AUC 0.948 y F1 0.812** (XGBoost); y (3) un **agente autónomo** que cierra el ciclo emitiendo 6 alertas proactivas priorizadas con traza completa. La **reproducibilidad** está respaldada por **seed = 42** en todo el pipeline y una suite de **12 pruebas unitarias**.

Limitaciones identificadas

Operación por ciclos. El sistema evalúa la red en **ciclos semanales**, alineado con la cadencia de decisión del negocio; los eventos que ocurren dentro de la semana se atienden en el siguiente ciclo de monitoreo, no en tiempo real. **Re-validación periódica.** Los modelos reflejan el régimen de demanda del histórico disponible; conviene **re-entrenarlos periódicamente** conforme evolucionen los patrones estacionales y de consumo para sostener la precisión en el tiempo. **Visualización.** El panel de flujos del Módulo 1 requiere una **interfaz gráfica**; en entornos sin entorno gráfico (servidores headless) debe configurarse una salida alternativa.

Trabajo futuro

Las prioridades de evolución, en orden de impacto: (1) **incorporar datos históricos reales** de ventas, inventario y entregas para re-calibrar los modelos y dar seguimiento a su desempeño en operación; (2) **ampliar la planeación a múltiples periodos** con inventario de seguridad y ventanas de tiempo, para anticipar reabastos con mayor horizonte; (3) **automatizar el re-entrenamiento** de los modelos y el seguimiento de **KPIs de negocio** como los desabastos evitados y el nivel de servicio; y (4) **escalar el agente a operación continua** con tableros en vivo para el equipo de logística. Cada paso es incremental sobre la arquitectura modular actual, sin reescribir el núcleo.